

5. 计算机的算术运算

■5.1 加法和减法

■5.2 乘法

■5.3 除法

■5.4 浮点运算

5.1 加法和减法

十进制加法，例：

$$\begin{array}{r} 245 \\ + 673 \\ \hline 918 \end{array}$$

加法表

0	0									
1	1	2								
2	2	3	4							
3	3	4	5	6						
4	4	5	6	7	8					
5	5	6	7	8	9	10				
6	6	7	8	9	10	11	12			
7	7	8	9	10	11	12	13	14		
8	8	9	10	11	12	13	14	15	16	
9	9	10	11	12	13	14	15	16	17	18
+	0	1	2	3	4	5	6	7	8	9

十进制加法

□ 十进制加法，例：

$$\begin{array}{r} 245 \\ + 673 \\ \hline 918 \end{array}$$

□ 加法表

□ 加法位表

□ 进位位表

加法位表

0	0									
1	1	2								
2	2	3	4							
3	3	4	5	6						
4	4	5	6	7	8					
5	5	6	7	8	9	0				
6	6	7	8	9	0	1	2			
7	7	8	9	0	1	2	3	4		
8	8	9	0	1	2	3	4	5	6	
9	9	0	1	2	3	4	5	6	7	8
+	0	1	2	3	4	5	6	7	8	9

进位位表

0	0									
1	0	0								
2	0	0	0							
3	0	0	0	0						
4	0	0	0	0	0					
5	0	0	0	0	0	1				
6	0	0	0	0	1	1	1			
7	0	0	0	1	1	1	1	1		
8	0	0	1	1	1	1	1	1	1	
9	0	1	1	1	1	1	1	1	1	1
+	0	1	2	3	4	5	6	7	8	9

二进制加法

二进制加法表(真值表)

0	00	01
1	01	10
+	0	1

0	0	1
1	1	0
+	0	1

0	0	0
1	0	1
+	0	1

加法位表

进位位表

二进制加法示例：

有符号怎么办？

	1	0	0	1	0	0	1	0	1
+	1	1	0	1	1	0	1	1	0
	?	?	?	?	?	?	?	?	?

无符号数加法

补码的表示

■ 符号部分同原码

- 数的最高位为符号位，0表示正数，1表示负数

■ 数值部分与它的符号位有关

- 对于正数，补码数值部分与原码数值部分相同
- 对于负数，补码数值部分是将原码数值部分按位取反再加1，即在反码数值部分基础上加1

$$X = +1101$$

$$[X]_{\text{原}} = 00001101$$

$$[X]_{\text{补}} = 00001101$$

$$Y = -1110$$

$$[Y]_{\text{原}} = 10001110$$

$$[Y]_{\text{补}} = 11110010$$

补码加减法

- 补码加法：符号位参与运算
- 补码减法：变为加法
- 采用补码进行加减法运算，在计算机中只需要一套实现加法运算的电路，从而简化了计算机内部硬件电路的结构

$$[A + B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}}$$

例1：A= 1011， B= -1110， 求 $[A+B]_{\text{补}}$

解： $[A]_{\text{补}}=0\ 1011$ ， $[B]_{\text{补}}=1\ 0010$

$$\begin{array}{r} 0\ 1011 \\ +\ 1\ 0010 \\ \hline \end{array}$$

$$[A+B]_{\text{补}}=1\ 1101 \qquad 1\ 1101$$

$$[A - B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}}$$

例1：A= 1011， B= -0010， 求 $[A-B]_{\text{补}}$

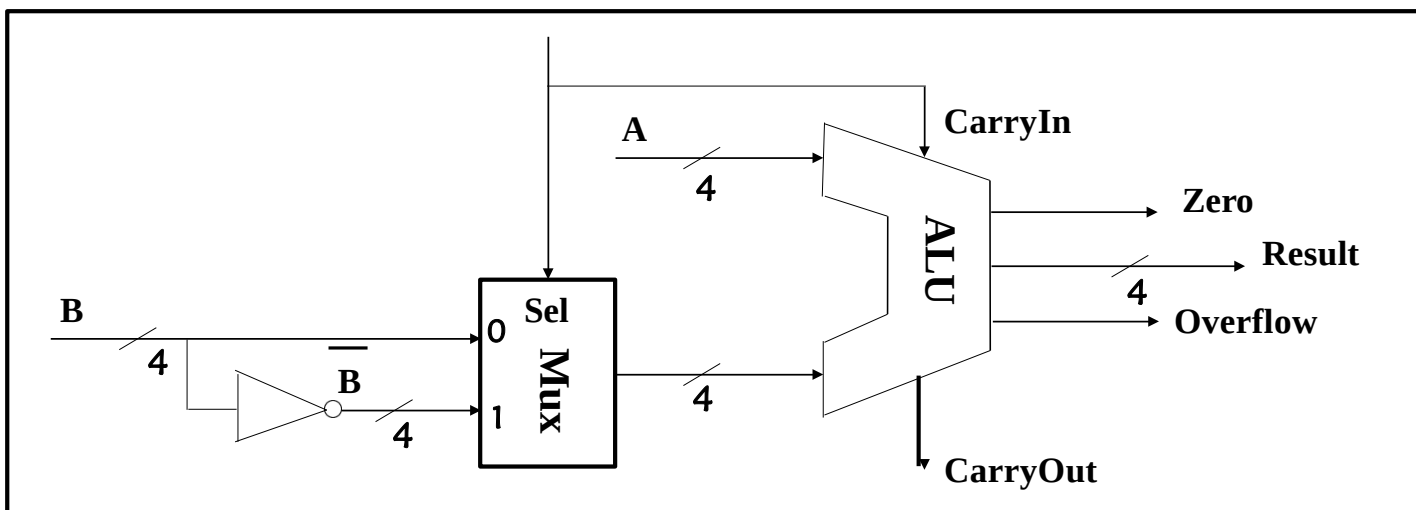
解： $[A]_{\text{补}}=0\ 1011$ ， $[-B]_{\text{补}}=0\ 0010$

$$\begin{array}{r} 0\ 1011 \\ +\ 0\ 0010 \\ \hline \end{array}$$

$$[A-B]_{\text{补}}=0\ 1101 \qquad 0\ 1101$$

补码加减法电路

- 加法运算和减法运算使用同一个加法电路，简化了运算器的设计



补码加减法的溢出

■ 补码运算示例（用5位二进制补码表示数）

$A=+1111$, $B=-1101$, 求 $A+B$ $[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}}$

$[A]_{\text{补}}=01111$, $[B]_{\text{补}}=10011$,

$$\begin{array}{r} 01111 \\ +) 10011 \\ \hline 100010 \end{array}$$

进位标志位 $CF=1$
溢出标志位 $OF=0$

$[A+B]_{\text{补}}=00010$

补码加减法的溢出

■ 补码运算示例（用5位二进制补码表示数）

例：A= -1111, B= - 1101, 求A+B

$$[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}}$$

$$[A]_{\text{补}} = 10001, [B]_{\text{补}} = 10011,$$

$$\begin{array}{r} 0 0 0 1 \\ +) 0 0 1 1 \\ \hline 1 0 0 1 0 0 \end{array}$$

进位标志位CF=1
溢出标志位OF=1

$$[A+B]_{\text{补}} = 00100$$

补码加减法的溢出

- 需要考虑溢出问题，即运算结果超出了机器能表示数的范围

例：A1 = + 1101, B1 = + 1001
A2 = - 1011, B2 = - 1100
求A1 + B1, A2 + B2

$$[A_1]_{\text{补}} + [B_1]_{\text{补}} = 01101 + 01001 = 10110$$

和大于最大表示的已整收

正溢出

$$[A_2]_{\text{补}} + [B_2]_{\text{补}} = 10101 + 10100 = 01001$$

和小于最小表示的已整收

负溢出

- 正溢出：两个正数相加得到一个负数结果。
- 负溢出：两个负数相加得到一个正数结果。

补码加减法的溢出

■ 溢出检测方法：变形补码，又称“模4补码”

- 用两个二进制位来表示数字的符号位，其余与补码相同。即，用“00”表示正，用“11”表示负。
- 进行加减运算时，当运算结果的符号位出现“01”或者“10”时，则表示产生溢出。
- 变形补码的最高位（第一个符号位）表示正确的符号，
 - “00”、“01” 分别表示正数、正溢出（上溢）
 - “11”、“10” 表示负数、负溢出（下溢）

符号位	相同	01	10
状态表示	正常	正溢出	负溢出

A = - 1101

B = - 1010

求 $[A+B]_{\text{补}}$

$[A]_{\text{补}} = 11\ 0011$

$[B]_{\text{补}} = 11\ 0110$

11 0011

+ 11 0110

10 1001

结果为负溢出

A = 1101

B = - 1010

求 $[A-B]_{\text{补}}$

$[A]_{\text{补}} = 00\ 1101$

$[-B]_{\text{补}} = 00\ 1010$

00 1101

+ 00 1010

01 0111

结果为正溢出

MIPS中的计算溢出问题

- 采用两种类型的算术指令
 - Add, addi, sub指令在溢出时产生异常
 - Addu, addiu, subu无符号数指令发生溢出时忽略, 不产生异常

5.2 乘法

■ 十进制乘法，例：

				2	4	5
	X			6	7	3
$M_0 = AXB_3$				7	3	5
$M_1 = AXB_2$		1	7	1	5	
$M_2 = AXB_1$	1	4	7	0		
$AXB = \sum 10^i M_i$	1	6	4	8	8	5

位积 $A \times B_i$

$$AXB = 164885$$

二进制乘法

逻辑与					1	1	0	1
	X				1	0	1	1
$M_0 = A \& B_4$					1	1	0	1
$M_1 = A \& B_3$			1	1	0	1		
$M_2 = A \& B_2$		0	0	0	0			
$M_3 = A \& B_1$	1	1	0	1				
$AXB = \sum 2^i M_i$	1	0	0	0	1	1	1	1

位积 $A \times B_i$

$$AXB = 10001111$$

二进制乘法

					1	1	0	1
				X	1	0	1	1
$M_0 = A \& B_4$	0	0	0	0	1	1	0	1
$M_1 = A \& B_3$	0	0	0	1	1	0	1	0
$M_2 = A \& B_2$	0	0	0	0	0	0	0	0
$M_3 = A \& B_1$	0	1	1	0	1	0	0	0
$AXB = \sum 2^i M_i$	1	0	0	0	1	1	1	1

手工运算过程

位积 $A \times B_i$

$AXB = 10001111$

计算机中怎么实现?

二进制乘法

[illegible]

被乘数每次左移一位,与前述和相加 $\Rightarrow$ 加法器

解决方案1：引入部分积P，改一次求和为累加求和

二进制乘法

部分积的最后一位不参与加法运算
直接右移出即可

部分积右移代替位积左移

只需要n位加法器

$$\begin{aligned}
 M_0 &= A \times B_4 & P_1 &= P_0 + 2^0 M_0 \\
 M_1 &= A \times B_3 & P_2 &= P_1 + 2^1 M_1 \\
 M_2 &= A \times B_2 & P_3 &= P_2 + 2^2 M_2 \\
 M_3 &= A \times B_1 & P_4 &= P_3 + 2^3 M_3
 \end{aligned}$$

AXB

$$\begin{array}{r}
 1101 \\
 \times 1011 \\
 \hline
 \end{array}$$

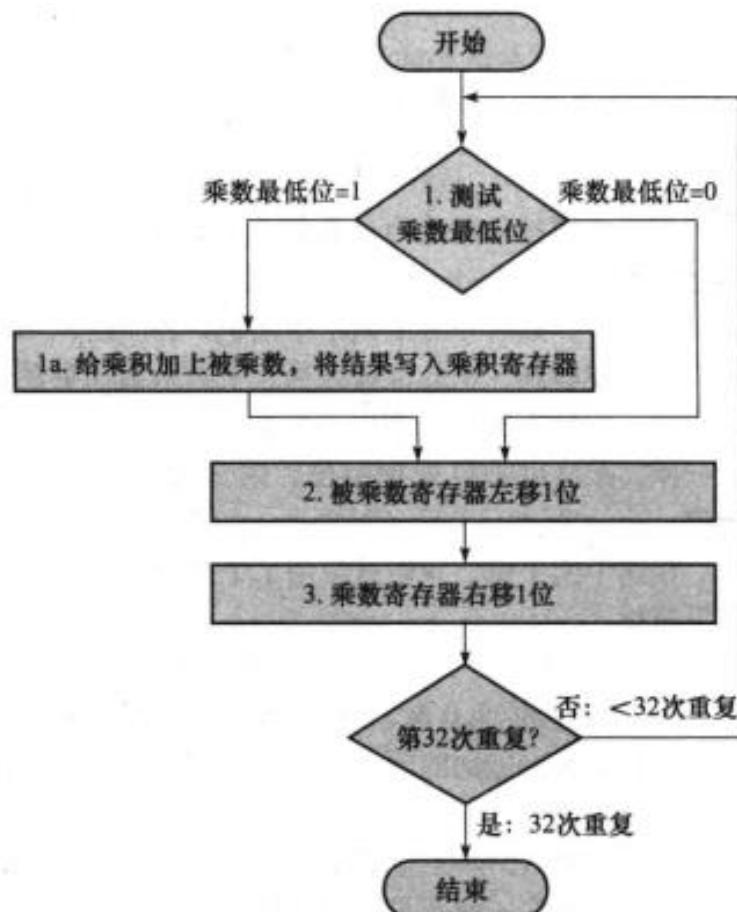
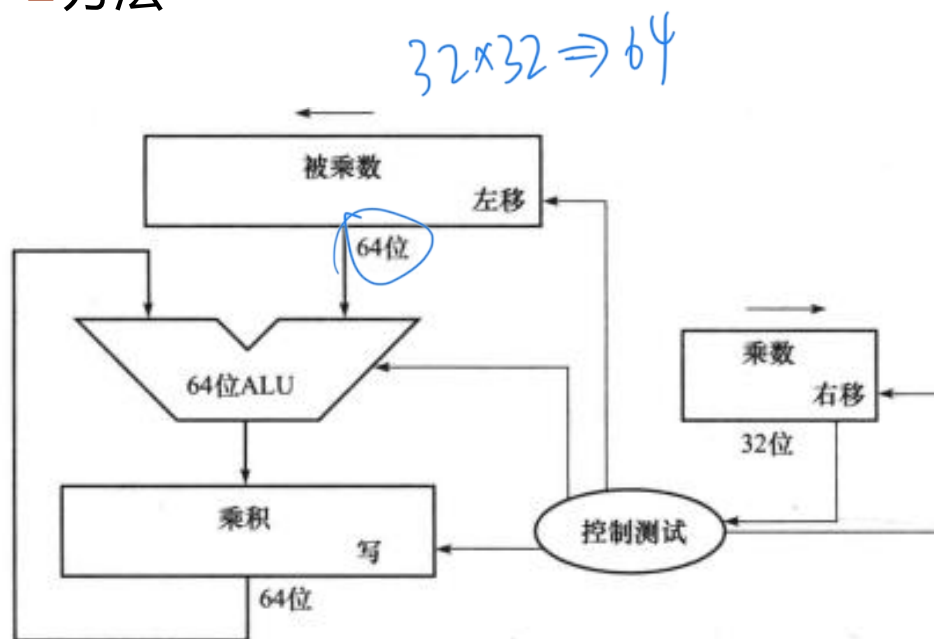
1	1	0	1
1	1	0	1

1 0 0 1 1 1

解决方案2：将部分积右移一位再求和，移出部分保存

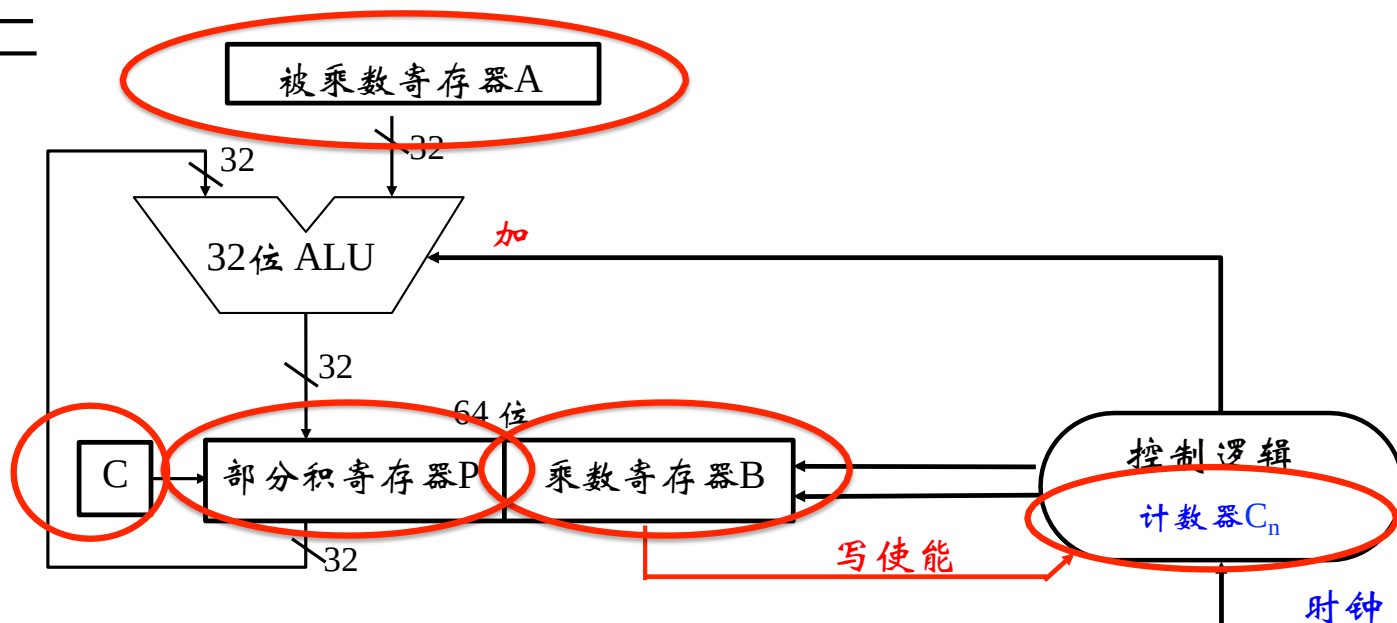
二进制无符号数乘法

■ 方法一



二进制无符号数乘法

■ 方法二



被乘数寄存器A: 存放被乘数

部分积寄存器P: 初始置 $P_0 = 0$; 结束时存放64位乘积的高32位

乘数寄存器B: 初始置乘数; 结束时存放64位乘积的低32位

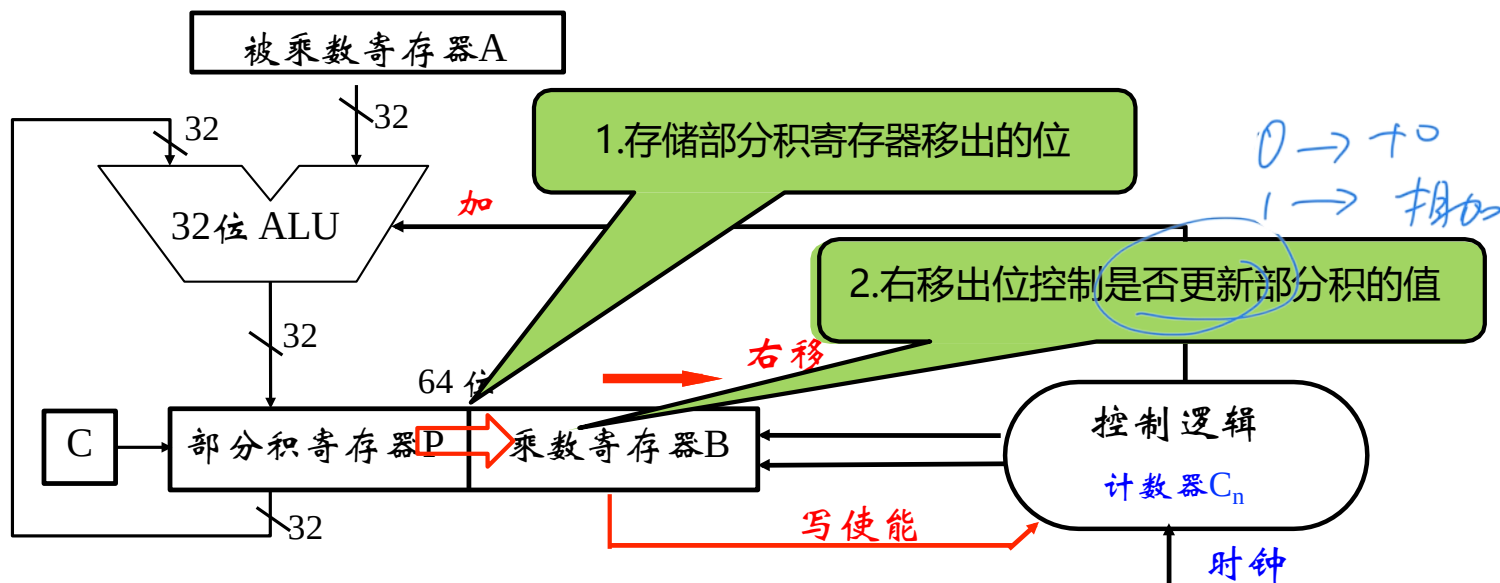
进位触发器C: 保存加法器的进位信号

循环计数器 C_n : 循环次数(初值32), 每循环一次 $C_n - 1$, $C_n = 0$ 结束

ALU: 乘法核心部件, 在“加”逻辑控制下对P和A的内容进行运算, 在“写使能”逻辑控制下加法运算结果被送回P, 进位存放在C中

二进制无符号数乘法

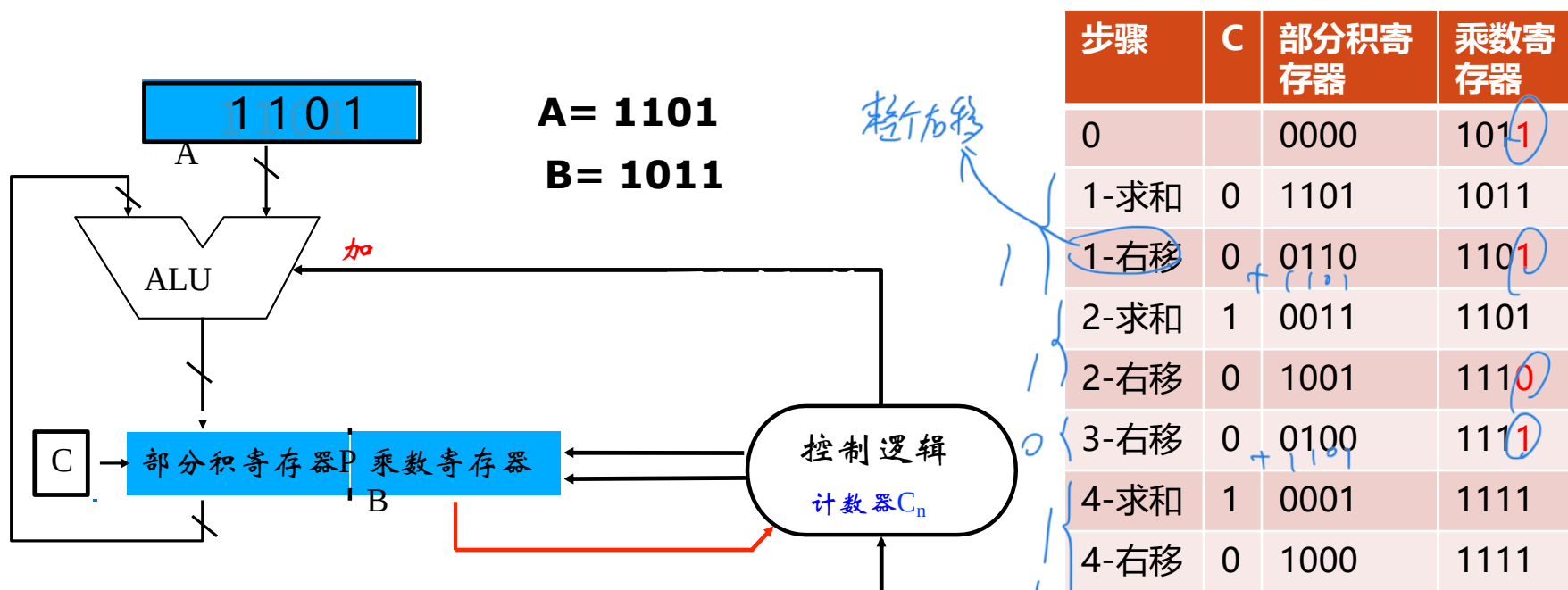
推荐这种方法



每次循环都要对进位C、部分积寄存器P和乘数寄存器B实现同步“右移”

- 乘法运算需要多次累加和移位操作
 - n位数的乘法需要n次加法和右移操作

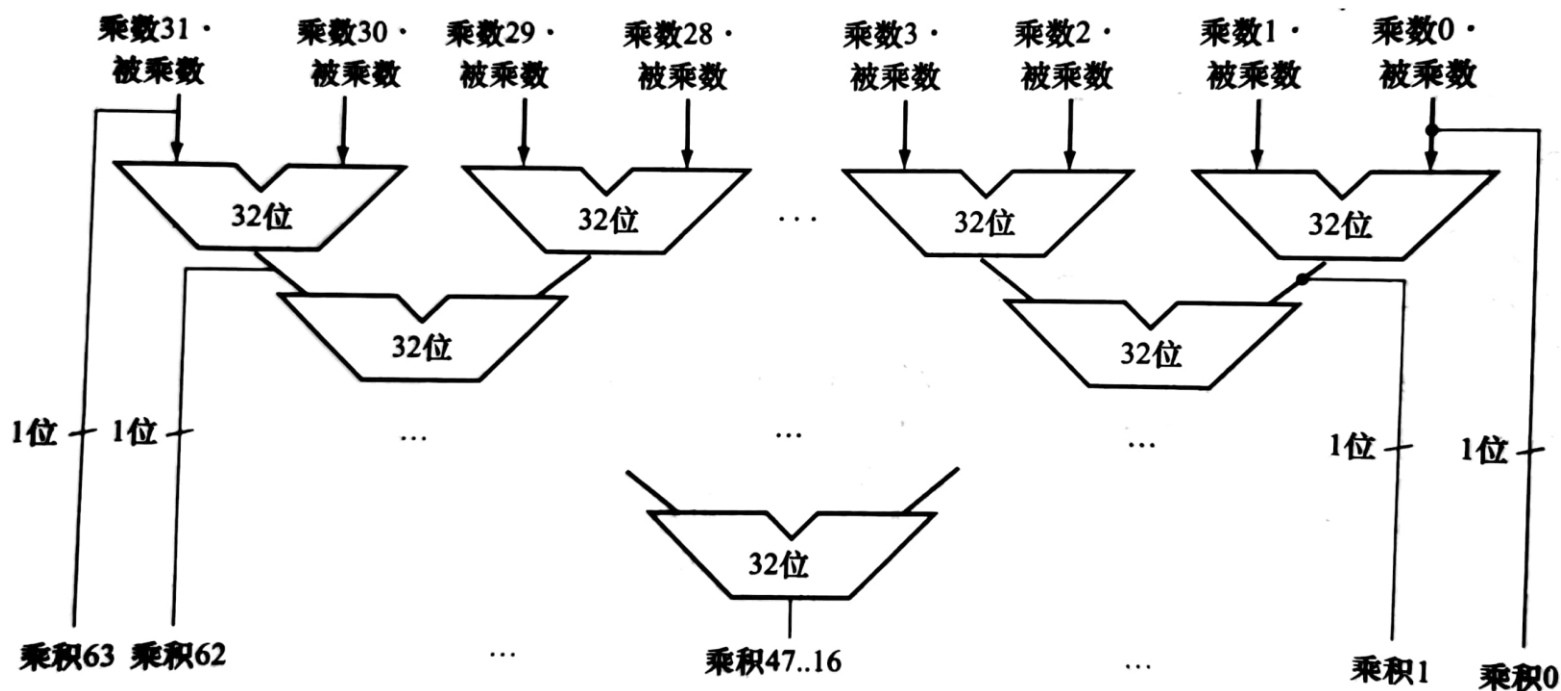
二进制无符号数乘法



- 最终乘积结果的高四位保存在部分积寄存器，低四位保存在乘数寄存器

更快速的乘法

- 32个加法器组成并行树，需要 $\log_2(32)$ 个加法时间单位



有符号数乘法

- 最简单的方法
 - 将被乘数和乘数转化为正数
 - 符号位不参加乘法运算，符号相异，积为负，
 - 符号位：被乘数和乘数符号位的异或

MIPS的乘法

- 一对特殊的32位寄存器：Hi, Lo
 - 乘法：存放64位的积Hi寄存器保存结果的高32位，Lo寄存器保存结果的低32位
 - 除法：HI寄存器保存余数，LO寄存器保存商
 - mfhi rd：将特殊寄存器HI的值赋给rd通用寄存器。
 - mflo rd：将特殊寄存器LO的值赋给rd通用寄存器。
 - mthi rs：将地址为rs的通用寄存器的值赋给特殊寄存器HI。
 - mtlo rs：将地址为rs的通用寄存器的值赋给特殊寄存器LO。
- 乘法指令：
 - mult, multu（无符号数乘法）

5.3 除法

■ 启发：十进制除法

- 除法是乘法的逆运算，本质是一种试探法
- 除法运算的基本思路：从被除数中试探减除数，通过判断余数是否够减，决定商的值
- 余数
 - 真余数：余数的符号位与被除数相同
 - 假余数：余数的符号位与被除数不同

$$\begin{array}{r} \text{商} \longrightarrow 6 \\ \text{除数} \longrightarrow 12 \overline{) 6521} \\ \text{被除数} \longrightarrow 6521 \\ \text{假余数} \longrightarrow -7 \\ \quad -72 \\ \hline \end{array}$$

$$\begin{array}{r} 12 \overline{) 6521} \\ \underline{-60} \\ 52 \\ \underline{-48} \\ 41 \\ \underline{-36} \\ 5 \end{array}$$

真余数 $\longrightarrow 5$

二进制除法

■ 二进制除法可模仿十进制除法运算

■ 只需要试探商1还是商0:

- 余数减去除数，够减则商1，否则商0

- 若商0，则多减了，需要恢复余数

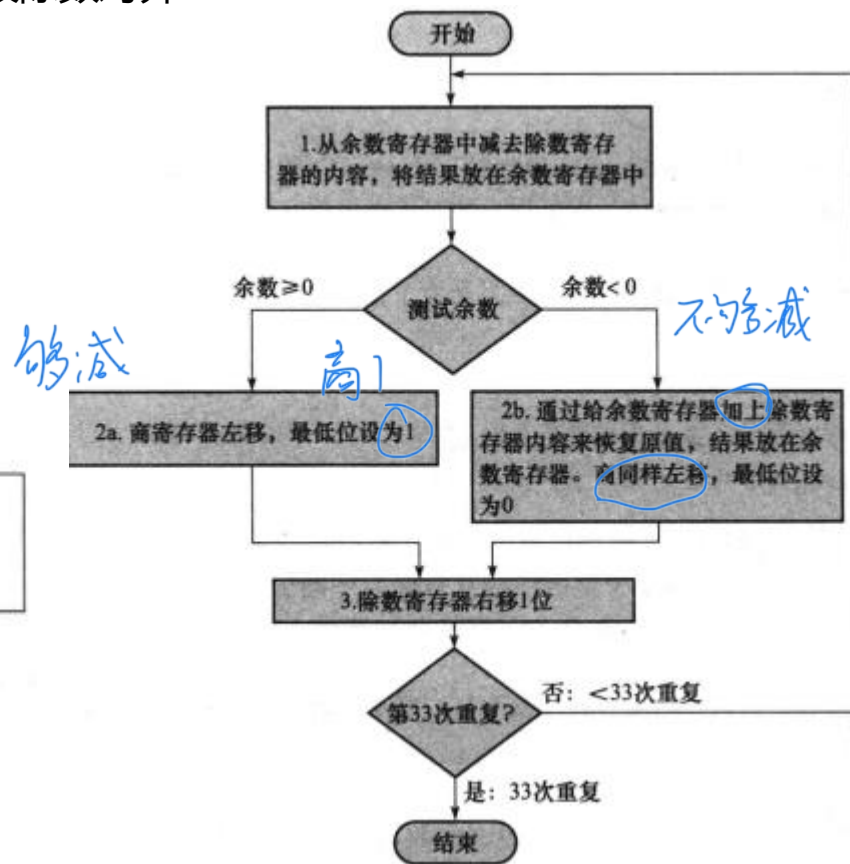
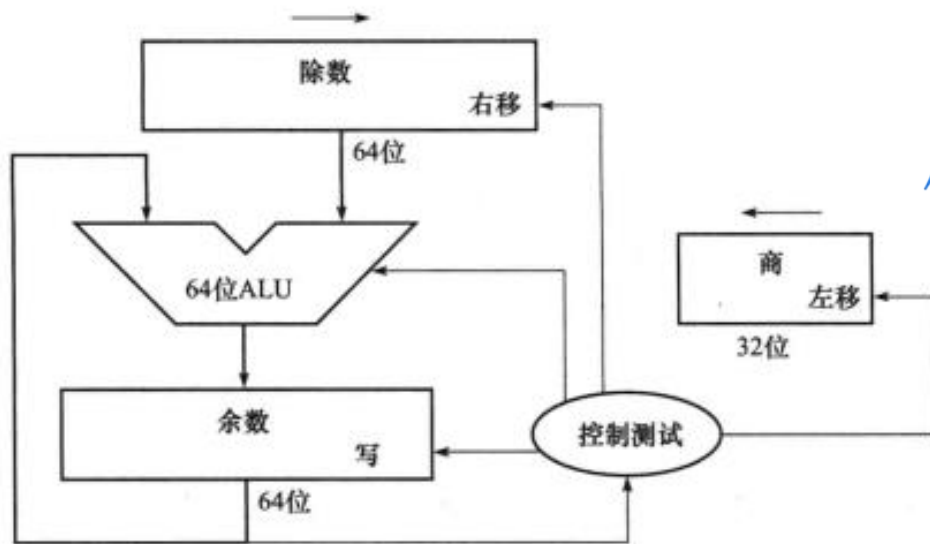
- 除数每次右移一位与余数比较： 等效于除数不动，而使余数左移一位

The diagram illustrates the binary division process. On the left, the divisor (除数) is 1011, enclosed in a red box. A purple arrow points from the label to the box. To the right of the divisor is a vertical line, followed by the dividend (被除数) 1101010, with a red arrow pointing from the label to the number. Above the dividend, the quotient (商) 1001 is written, with a blue arrow pointing from the label to the number. Below the dividend, the subtraction steps are shown: -1011 is subtracted from 1101010 to get 10010, and then -1011 is subtracted from 10010 to get the remainder 111. The remainder (余数) 111 is at the bottom, with a green arrow pointing from the label to the number. The subtraction steps are also enclosed in red boxes.

$$\begin{array}{r} 1001 \text{ (商)} \\ 1011 \overline{) 1101010 \text{ (被除数)}} \\ \underline{-1011} \\ 10010 \\ \underline{-1011} \\ 111 \text{ (余数)} \end{array}$$

二进制除法——方法一

- 初始，除数占用寄存器的左半边，每次右移一位与被除数对齐
- 余数寄存器初始化为被除数



例: $7_{10} \div 2_{10} = 0000\ 0111_2 \div 0010_2$

除数4位, 循环5次

被除数初始放在
余数reg

迭代次数	步骤	商	除数	余数
0	初始值	0000	0010 0000	0000 0111
1	1: 余数=余数-除数	0000	0010 0000	①110 0111
	2b: 余数<0 $\Rightarrow$ +除数, 商左移, 上最低位上0	0000	0010 0000	0000 0111
	3: 除数右移	0000	0001 0000	0000 0111
	1: 余数=余数-除数	0000	0001 0000	①111 0111
	2b: 余数<0 $\Rightarrow$ +除数, 商左移, 上最低位上0	0000	0001 0000	0000 0111
	3: 除数右移	0000	0000 1000	0000 0111
3	1: 余数=余数-除数	0000	0000 1000	①111 1111
	2b: 余数<0 $\Rightarrow$ +除数, 商左移, 上最低位上0	0000	0000 1000	0000 0111
	3: 除数右移	0000	0000 0100	0000 0111
4	1: 余数=余数-除数	0000	0000 0100	①000 0011
	2a: 余数 $\geq 0 \Rightarrow$ 商左移, 上最低位上1	0001	0000 0100	0000 0011
	3: 除数右移	0001	0000 0010	0000 0011
5	1: 余数=余数-除数	0001	0000 0010	①000 0001
	2a: 余数 $\geq 0 \Rightarrow$ 商左移, 上最低位上1	0011	0000 0010	0000 0001
	3: 除数右移	0011	0000 0001	0000 0001

① ALU: 减法

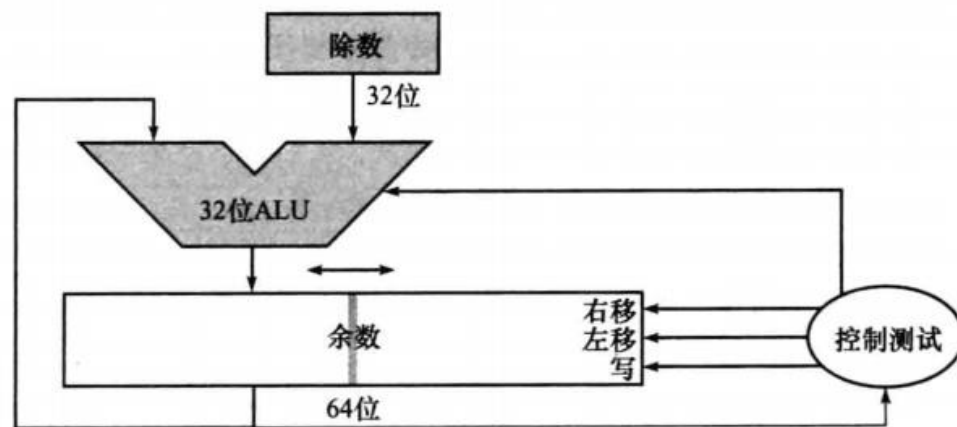
② 判断是否够减

商1
商0. 恢复余数

③ 除数右移

二进制除法——方法二

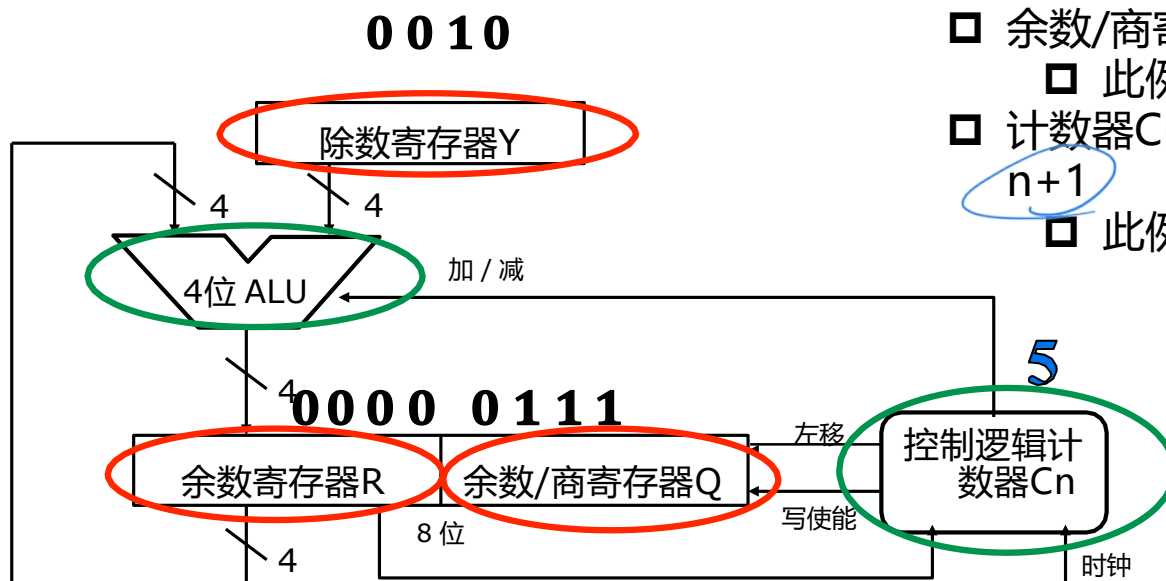
- ALU和除数寄存器位长减半
- 余数左移替代除数右移,
- 余数和商寄存器拼接存放初始的被除数, 余数存放被除数高位, 商存放被除数低位



乘法时部分积右移
除法时余数左移

二进制除法-方法二

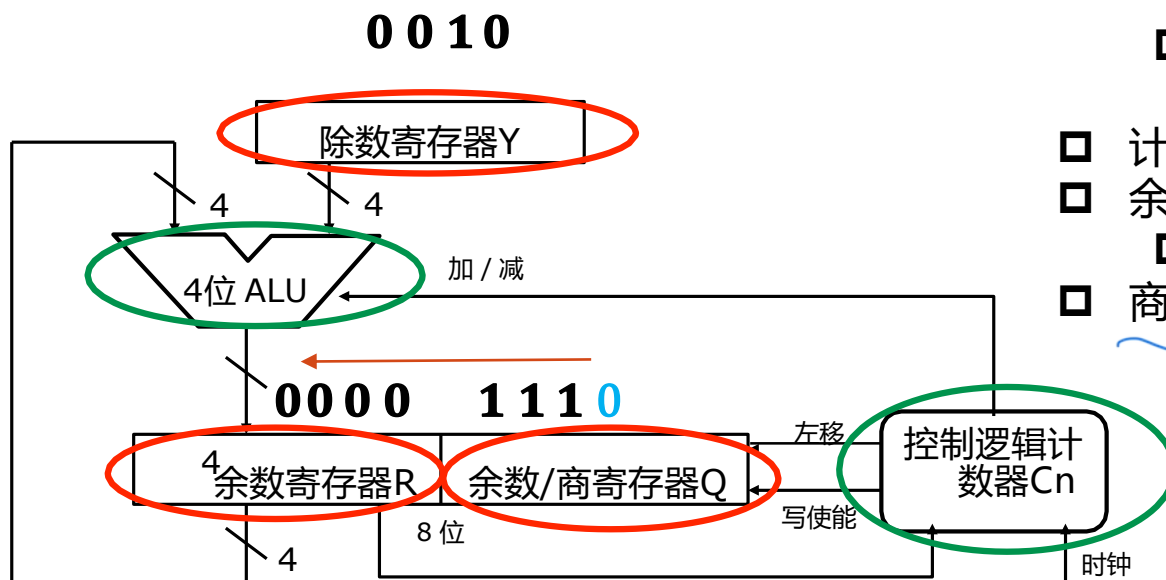
■ 例： $7_{10} \div 2_{10} = 0111_2 \div 0010_2$



- ① 初始状态
 - 余数寄存器R：存放被除数高位
 - 此例：0000；
 - 余数/商寄存器Q：存放被除数低位
 - 此例：被除数
 - 计数器Cn：需要循环运算的次数，初始
 - n+1
 - 此例：5

二进制除法-方法二

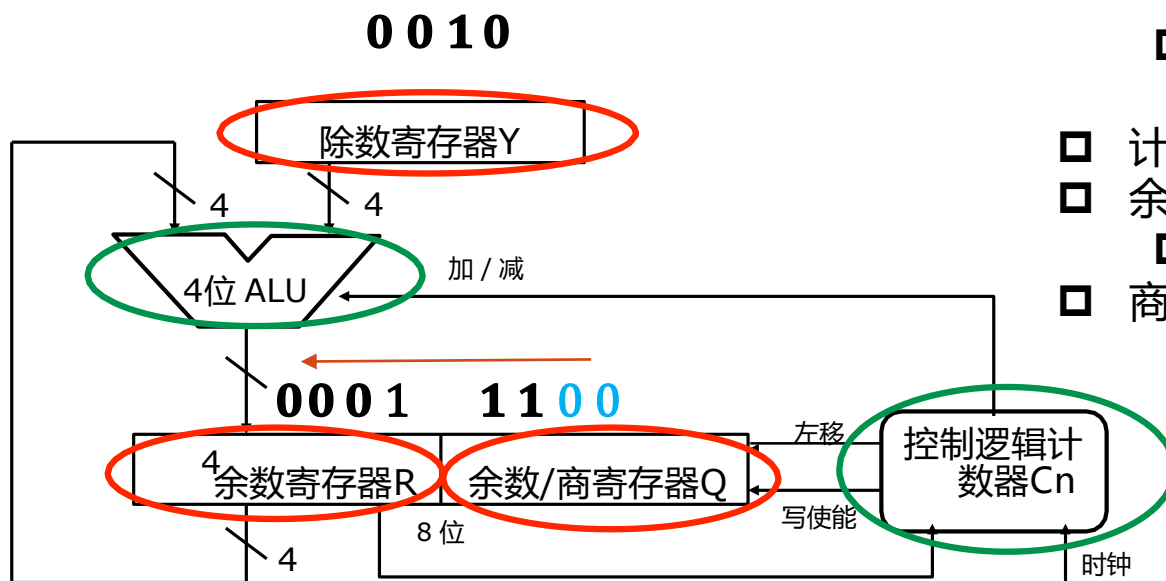
■ 例: $7_{10} \div 2_{10} = 0111_2 \div 0010_2$



- ② 第一次循环 (RQ: 0000 0111)
- ALU做余数-除数: 0000-0010
 - 差1110, CF标志为1, 有借位, 判断为不够减
 - 恢复余数: 余数+除数
1110+0010=0000
 - 计数器Cn减1: 5-1=4
 - 余数寄存器RQ左移1位
 - 00000111→00001110
 - 商0: 放入Q末尾

二进制除法-方法二

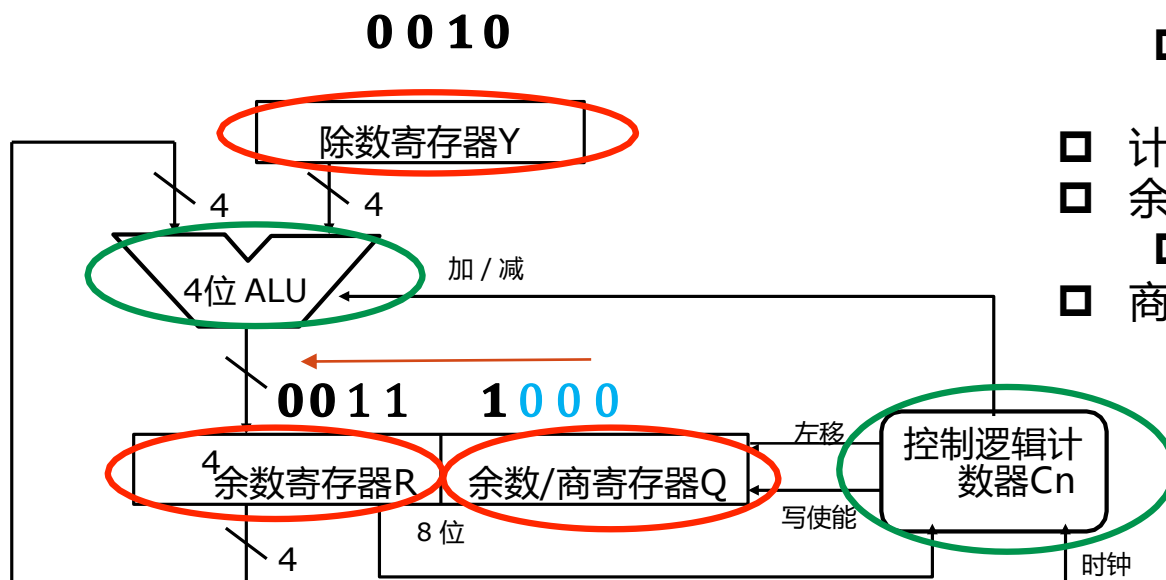
■ 例: $7_{10} \div 2_{10} = 0111_2 \div 0010_2$



- ③ 第二次循环 (RQ: 0000 1110)
 - ALU做余数-除数: 0000-0010
 - 差1110, CF标志为1, 有借位, 判断为不够减
 - 恢复余数: 余数+除数
1110+0010=0000
 - 计数器Cn减1: 4-1=3
 - 余数寄存器RQ左移1位
 - 00001110→00011100
 - 商0: 放入Q末尾

二进制除法-方法二

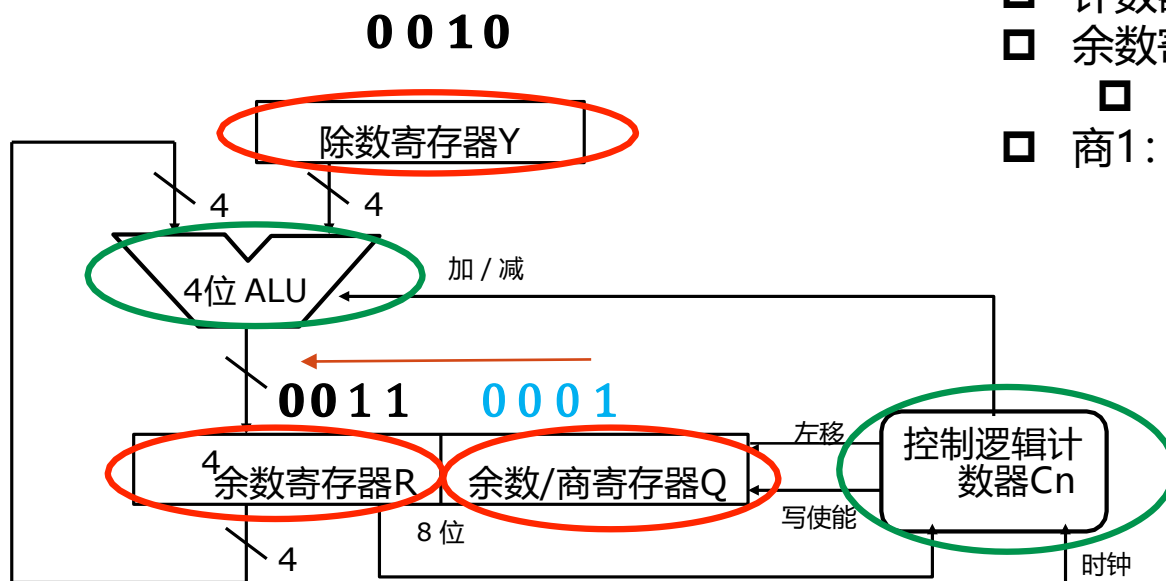
■ 例: $7_{10} \div 2_{10} = 0111_2 \div 0010_2$



- ④ 第三次循环 (RQ: 0001 1100)
 - ALU做余数-除数: 0001-0010
 - 差1111, CF标志为1, 有借位, 判断为不够减
 - 恢复余数: 余数+除数
1111+0010=0001
 - 计数器Cn减1: 3-1=2
 - 余数寄存器RQ左移1位
 - 00011100→00111000
 - 商0: 放入Q末尾

二进制除法-方法二

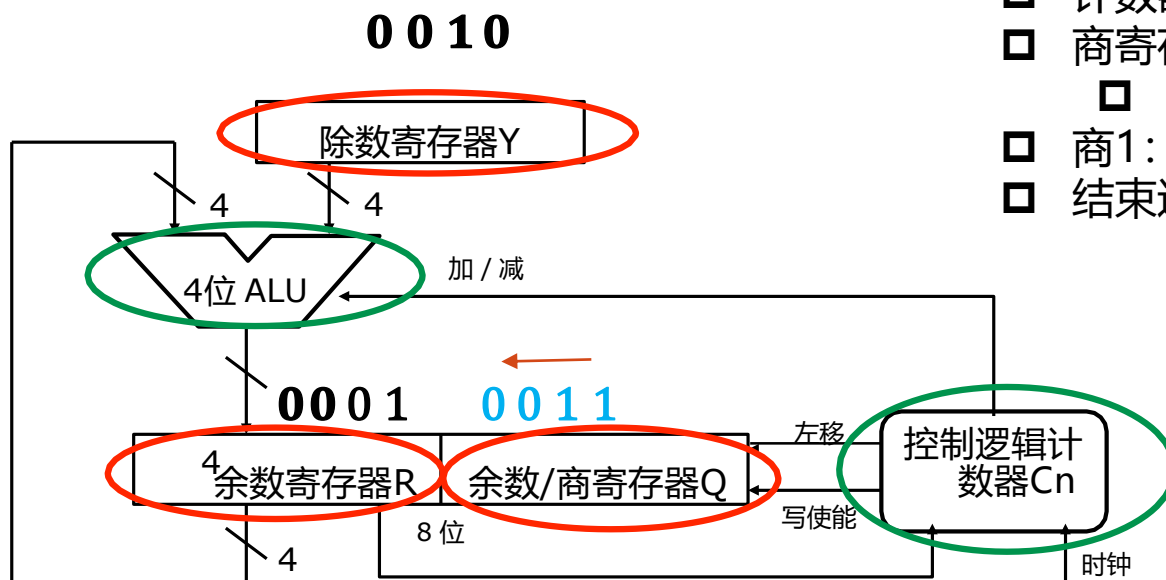
■ 例： $7_{10} \div 2_{10} = 0111_2 \div 0010_2$



- ⑤ 第四次循环 (RQ: 0011 1000)
 - ALU做余数-除数: 0011-0010
 - 差R0001, CF标志为0, 够减
 - 计数器Cn减1: $2-1=1$
 - 余数寄存器RQ左移1位
 - 00011000→00110000
 - 商1: 放入Q末尾 (RQ: 0011 0001)

二进制除法-方法二

■例: $7_{10} \div 2_{10} = 0111_2 \div 0010_2$



- ⑥ 第五次循环 (RQ: 0011 0001)
 - ❑ ALU做余数-除数: 0011-0010
 - ❑ 差R0001, CF标志为0, 够减
 - ❑ 计数器Cn减1: $1-1=0$,
 - ❑ 商寄存器Q左移1位, R不移位
 - ❑ 0001→ 0010
 - ❑ 商1: 放入Q末尾 (RQ: 0001 0011)
 - ❑ 结束运算

原码除法

- 将有符号的被除数 $[A]_{\text{原}}$ 和除数 $[B]_{\text{原}}$ 转换成正数(即绝对值 $|A|$ 和 $|B|$ 相除)
- 符号位单独处理, 商符号位 = 被除数和除数符号位做异或操作
- 余数 $[R]_{\text{原}}$ 的符号位必须与被除数的符号位保持一致

5.4 浮点运算

- 浮点表示
- 浮点加法运算
- 浮点乘除运算
- 算术运算的精确性

浮点数运算的相关问题

- Representation(表示)

- Normalized form (规格化形式) 和 Denormalized form (非规格化形式)

- 单精度和 双精度格式

- Range and Precision(表示范围和精度)

- Arithmetic (运算: $+$ 、 $-$ 、 $\times$ 、 $\div$)

- Rounding (舍入)

- Exceptions (异常处理)

- Errors (误差) 与精度控制

浮点数的表示

- 一般的浮点数表示为

$$(-1)^S \times F \times 2^E$$

符号位 尾数 指数

- IEEE 754标准：规格化的尾数整数部分为1

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

符号位 隐藏位 尾数的小数部分 基数 阶码 偏阶

$$-1.01101_2 \times 2^{10}$$

符号 尾数 阶码 基数

固定值，目的是希望最小的指数表示为0，最大的指数为全1

浮点数的表示

IEEE 754单精度表示

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
S	Exponent								Fraction																						

指数 2^7 . $\text{exp} = 2 + \text{bias} = 127$

Bias = 127

IEEE 754双精度表示

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48	47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
S	Exponent											Fraction																			
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Fraction																															

Bias = 1023

浮点数的表示示例

IEEE 754单精度表示

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	Fraction																						

2 (实际阶码) + 127 (bias) = 129 (阶码)

0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---

最小值 1

1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---

最大值 254

$$\text{exp}_{\max} = 2^8 - 2 = 254$$

$$\text{exp} - \text{bias} = 127$$

$$- 1.01101_2 \times 2^{10}$$

$$\text{bias} = 2^{n-1} - 1 \quad (n \text{ 为 exp 位数})$$

$$[1-127, 254-127]$$

$$[-126, 127]$$

浮点数的表示示例

IEEE 754单精度表示

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Fraction

$$-1.01101_2 \times 2^{10}$$

(10)₂
 $\text{exp} = 2 + \text{bias} = 129$

浮点数的表示

IEEE 754浮点数的编码表示

阶码全0和全1用作特殊值处理

单精度		双精度		表示的数
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	非0	0	非0	±非规格化数
1~254	任意	1~2046	任意	±浮点数
255	0	2047	0	±无穷
255	非0	2047	非0	NaN (非数)

浮点数的表示

■ IEEE754标准规定的五种异常情况

① 无效操作(无意义)

- 操作中有一个数是非有限数, 如: 加 / 减 ∞ 、 $0 \times \infty$ 、 $\infty \div y$ 等
- 结果无效, 如: 源操作数是NaN等

② 除以0(即: 无穷大)

③ 数太大(阶码上溢)

- 如: 对于单精度, 阶码 $E > 1111\ 1110$ (指数大于127)

④ 数太小(阶码下溢)

- 如: 对于单精度, 阶码 $E < 0000\ 0001$ (指数小于-126)

⑤ 结果不精确(舍入引起)

- 如: $1/3$ 不能精确表示成浮点数

异常情况硬件可以检测到, 可设定由硬件/软件处理

浮点数加法

■ 十进制科学计数法的加法

例如： $A=1.23 \times 10^5$ ； $B=4.56 \times 10^2$ ； 求 $A+B$

其计算过程为：

$$\begin{aligned} & 1.23 \times 10^5 + 4.56 \times 10^2 \\ = & 1.23 \times 10^5 + 0.00456 \times 10^5 \\ = & (1.23 + 0.00456) \times 10^5 = 1.23456 \times 10^5 \end{aligned}$$

进行尾数加法运算前，必须先“对阶”！

浮点数加法

- 对阶：使两个操作数的阶码相等
 - 规则：小阶向大阶看齐，阶小的数的尾数右移，右移位数等于两个阶码差的绝对值。
- IEEE754尾数右移时，需要注意
 - 要将隐含的“1”移到小数部分，空出位补0
 - 移出的低位保留到特定的“附加位”上

浮点数加法

■ 假设两个规格化浮点数分别为A和B

- $A = M_a \cdot 2^{E_a}$, $B = M_b \cdot 2^{E_b}$
- 假设 $E_a \geq E_b$

■ 浮点数加法步骤

1. 求阶差: $E_a - E_b$
2. 对阶: $M_b \cdot 2^{-(E_a - E_b)}$
3. 尾数相加: $M_a + M_b \cdot 2^{-(E_a - E_b)}$
4. 结果规格化: $A + B = (M_a + M_b \cdot 2^{-(E_a - E_b)}) \cdot 2^{E_a}$

浮点数运算结果溢出

- 溢出

- 阶码上溢

- 阶码下溢

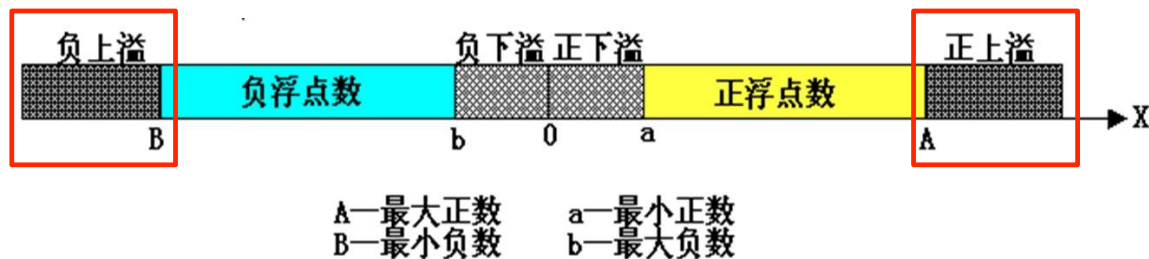
- 尾数上溢

- 尾数下溢

浮点数运算溢出

■ 阶码上溢:

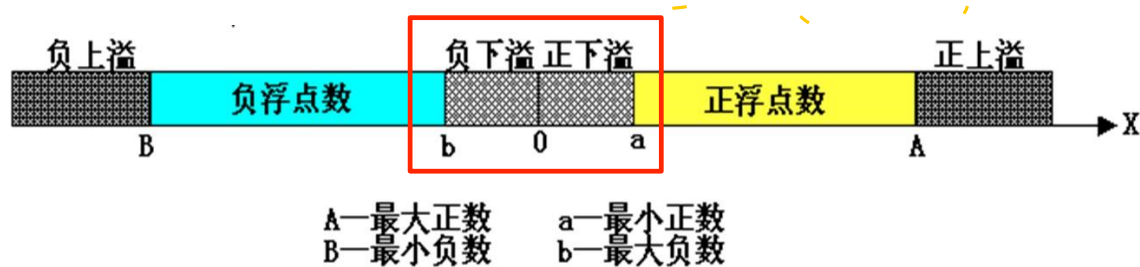
- 超过了阶码可以表示的最大允许值, 一般将其认为是 $+\infty$ 和 $-\infty$
- 单精度最大允许的阶码为 127



浮点数运算溢出

■ 阶码下溢

- 超过了阶码可能表示的最小允许值, 一般将其认为是0
- 单精度最小允许的阶码为: -126



浮点数运算溢出

■ 尾数上溢

- 两个同 / 异符号尾数相加 / 减，最高有效位产生了进位

■ 尾数溢出：

- 浮点数不一定溢出，即不一定会发生“异常”
- 解决方法：将尾数右移，阶码增1来重新对齐

$$\begin{array}{r} 1.010...0_2 \times 2^2 \\ + 1.100...0_2 \times 2^2 \\ \hline = 10.110...0_2 \times 2^2 \\ \Rightarrow 1.0110...0_2 \times 2^3 \end{array}$$

浮点数运算溢出

- 尾数下溢：在将尾数右移时，尾数的最低有效位从尾数域右端移出去，丢失了有效信息
 - 进行舍入处理
 - 在运算过程中，添加保护位

运算结果规格化

■浮点数加法步骤

1. 求阶差
2. 对阶
3. 尾数相加
4. 结果规格化及溢出处理
 - 舍入：如果尾数比规定位数长，需考虑舍入
 - 置0：尾数为0说明结果也为0，根据IEEE754，阶码和尾数全为0

运算结果规格化

■ 舍入

➤ **就近舍入**: 舍入为最近可表示的数

➤ 附加位

➤ 11: 入

➤ 01: 舍

➤ 10 (强制结果为偶数)

➤ 00: 保持结果不变

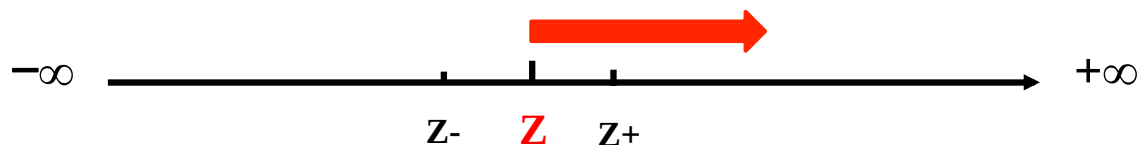
例: $1.1101\textcolor{red}{11} \sim 1.1110$; $1.1101\textcolor{red}{01} \sim 1.1101$;
 $1.1101\textcolor{red}{10} \sim 1.1110$; $1.1111\textcolor{red}{00} \sim 1.1111$

奇变 偶
若 10 不变 100.

Z-和Z+分别是结果 Z 的最近可表示的左、右数

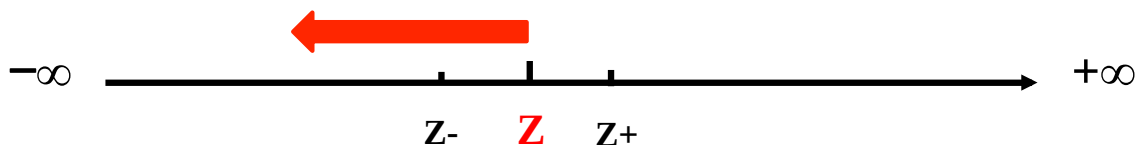
➤ 朝 $+\infty$ 方向舍入

➤ 舍入为Z+(正向舍入)



➤ 朝 $-\infty$ 方向舍入

➤ 舍入为Z-(负向舍入)



➤ 朝0方向舍入

➤ 总是舍入成Z+与Z-中绝对值较小的数 (更接近于0)



浮点加法运算示例

■ 例：用IEEE 754单精度形式，求出浮点数 $X=0.5_{10}$ 与 $Y=-0.4375_{10}$ 之和

解： $0.5_{10} = 1/2_{10} = 0.1_2 = (1.00...0_2) \times 2^{-1}$

$-0.4375_{10} = -7/16_{10} = -0.0111_2 = -1.110_2 \times 2^{-2}$

$[0.5]_{\text{浮}} = 0 \text{ 01111110 000...0}, [-0.4375]_{\text{浮}} = 1 \text{ 01111101 110...0}$

↓
符号位
↓
阶码

↓
尾数

↓
符号位
↓
阶码

↓
尾数

浮点加法运算示例 (续)

■ 例：用IEEE 754单精度形式，求出浮点数 $X=0.5_{10}$ 与 $Y=-0.4375_{10}$ 之和

解： $[0.5]_{\text{浮}} = 0\ 01111110\ 000...0$, $[-0.4375]_{\text{浮}} = 1\ 01111101\ 110...0$

对阶 (求阶差)：

$$\begin{aligned} [\Delta E]_{\text{补}} &= [E_x]_{\text{移}} - [E_y]_{\text{移}} \\ &= [E_x]_{\text{移}} + [-E_y]_{\text{移}}_{\text{补}} \pmod{256} \end{aligned}$$

$$[\Delta E]_{\text{补}} = 0111\ 1110 + 1000\ 0011 = 0000\ 0001, \Delta E = 1$$

所以：对Y进行对阶， $[Y]_{\text{浮}} = 1\ 0111\ 1110\ 1110...0$

尾数相加： $01.0000...0 + 10.1110...0 = 00.00100...0$

尾数相加要算上整数部分的隐含位

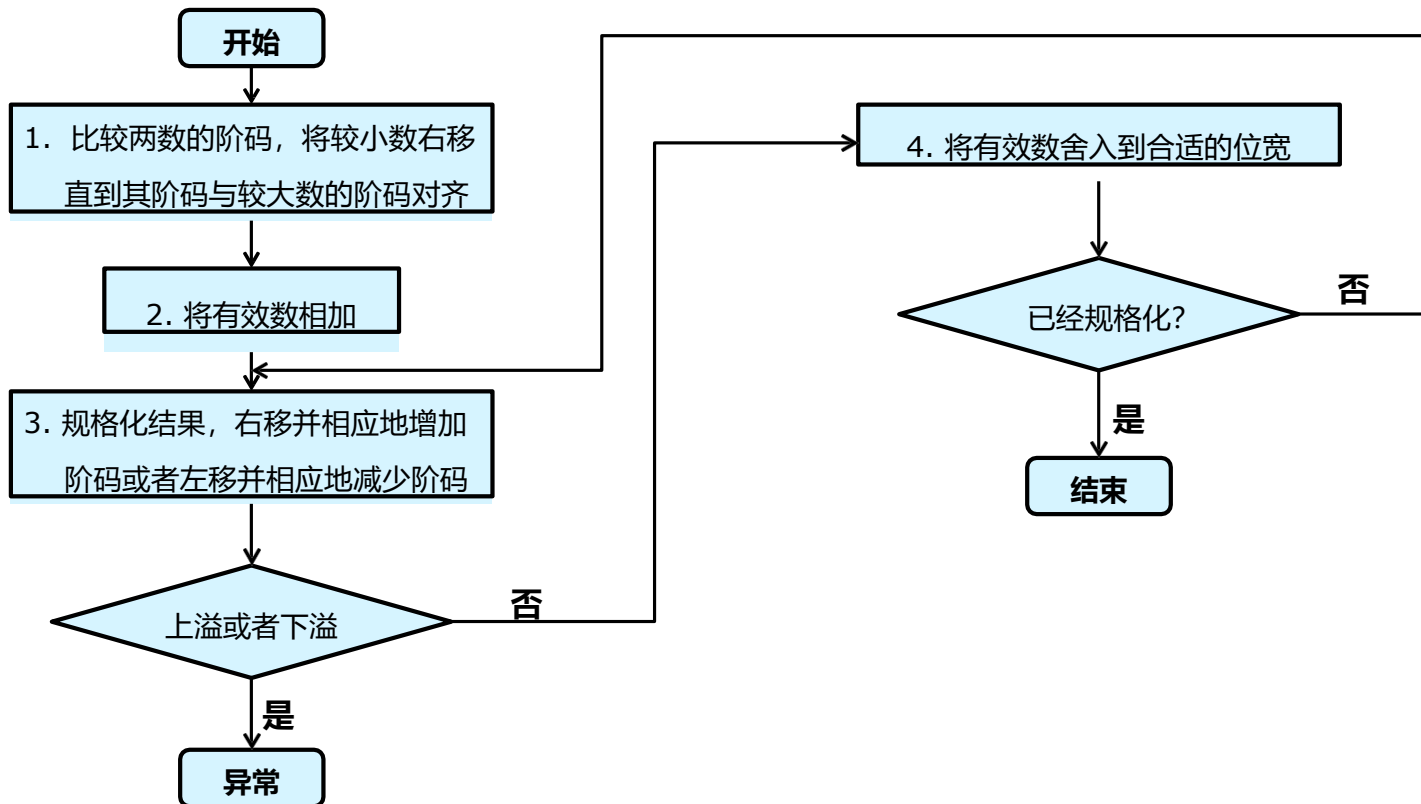
两个异号的原码数加法
(尾数为原码表示，最左边一位为符号位)

规格化和判溢出： $+(0.00100...0)_2 \times 2^{-1} = +(1.00...0)_2 \times 2^{-4}$ (阶码减3)

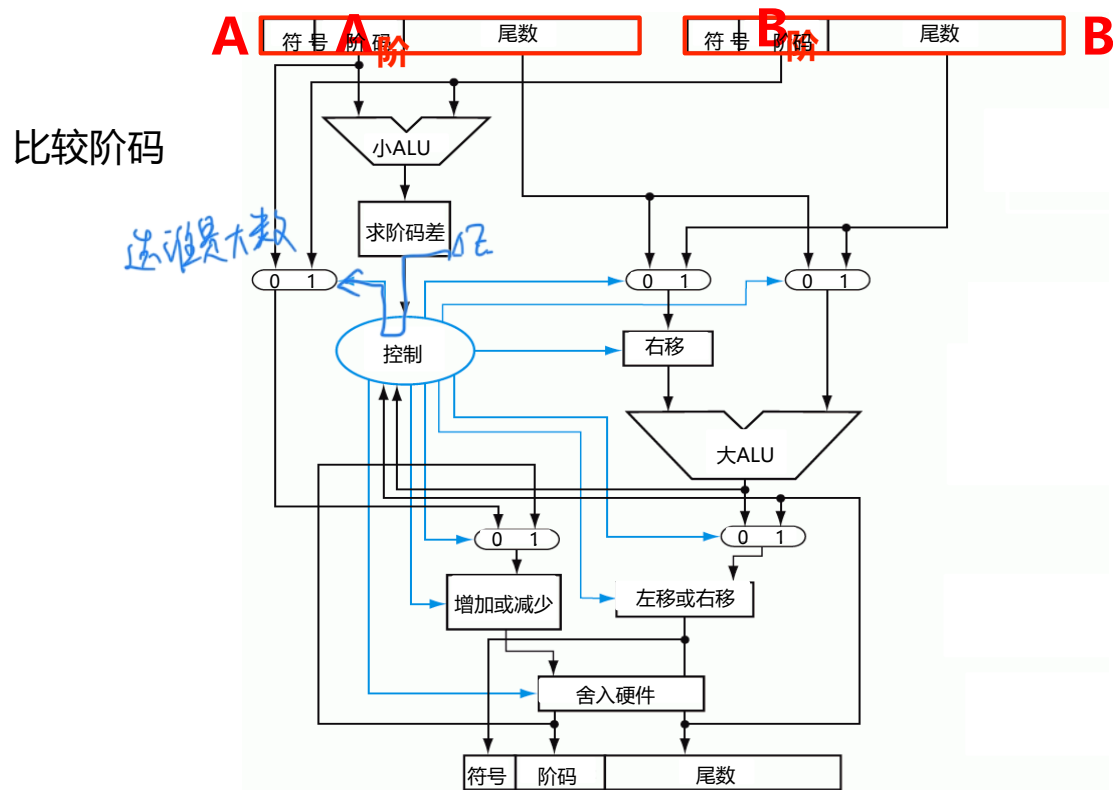
因为 $127 \geq -4 \geq -126$ ，没有溢出

舍入：无需舍入

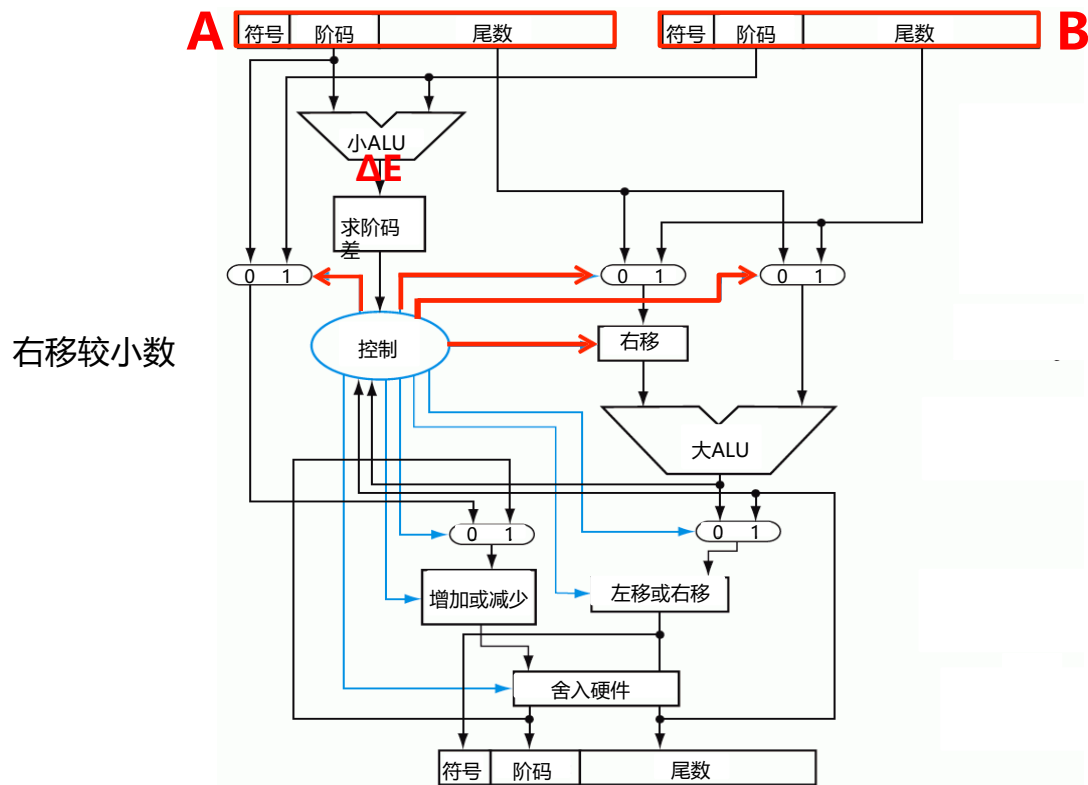
浮点加法运算



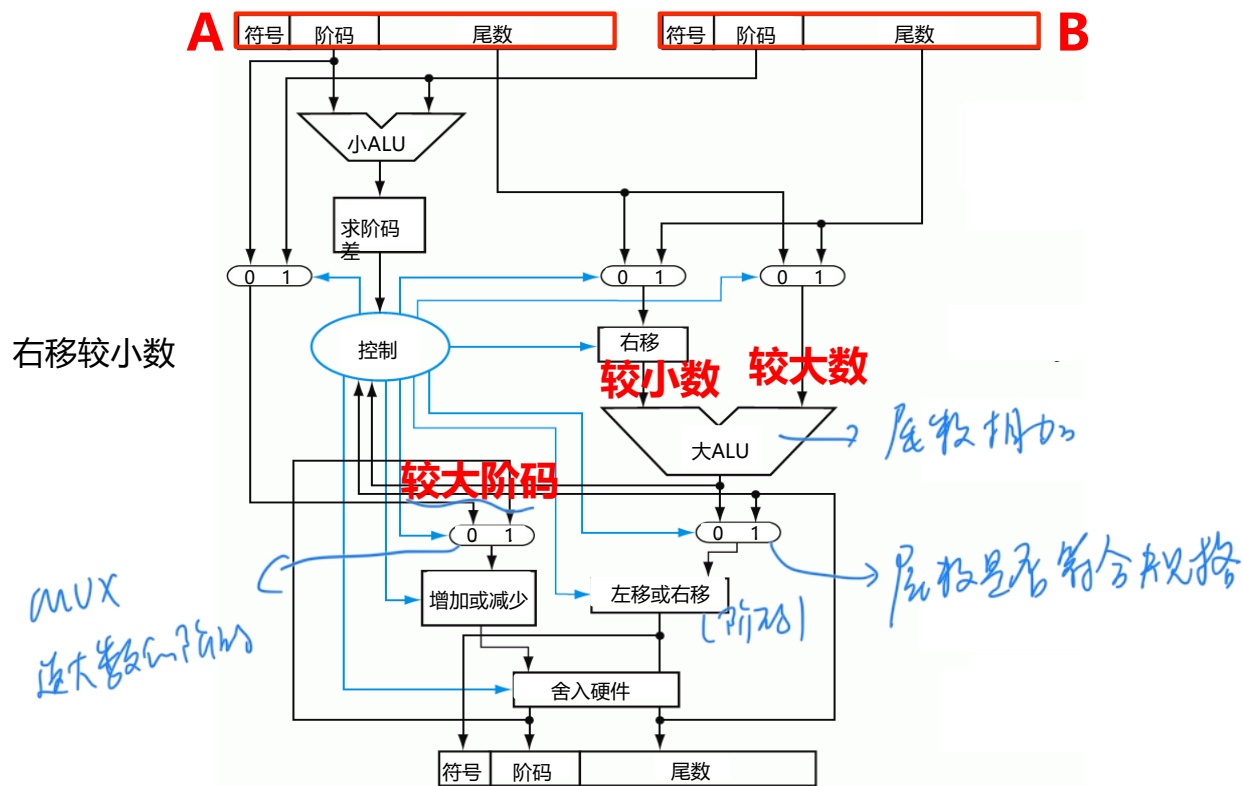
浮点加法硬件结构



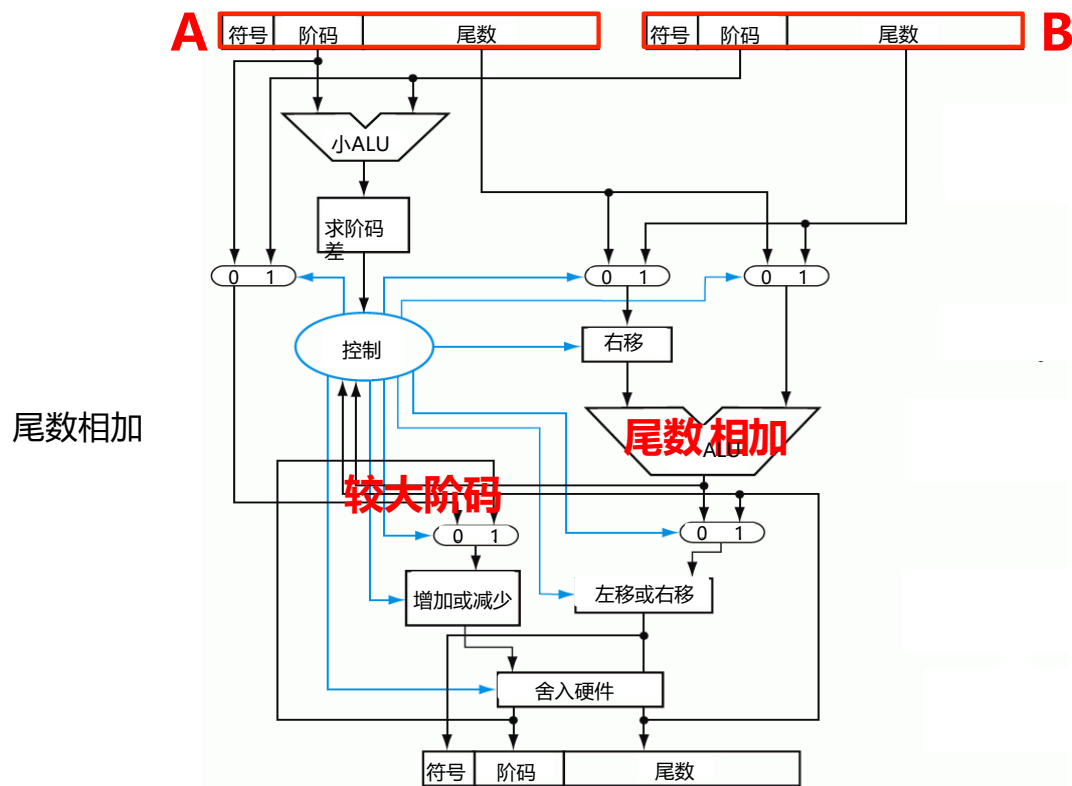
浮点加法硬件结构



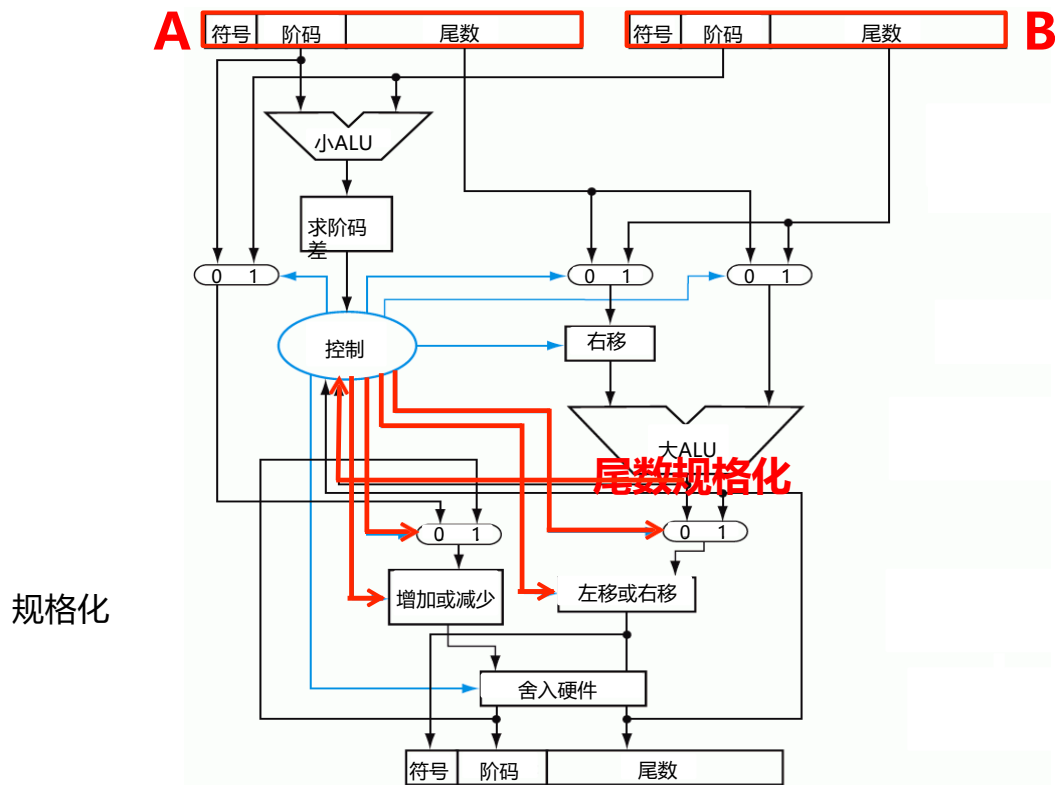
浮点加法硬件结构



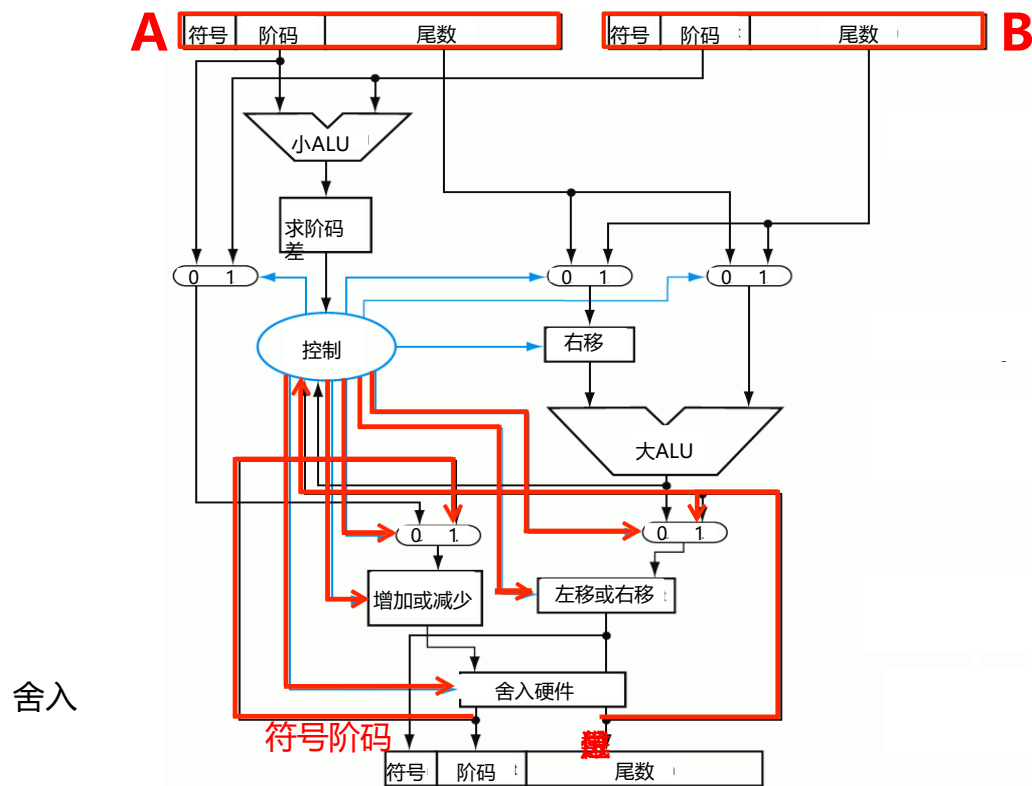
浮点加法硬件结构



浮点加法硬件结构



浮点加法硬件结构



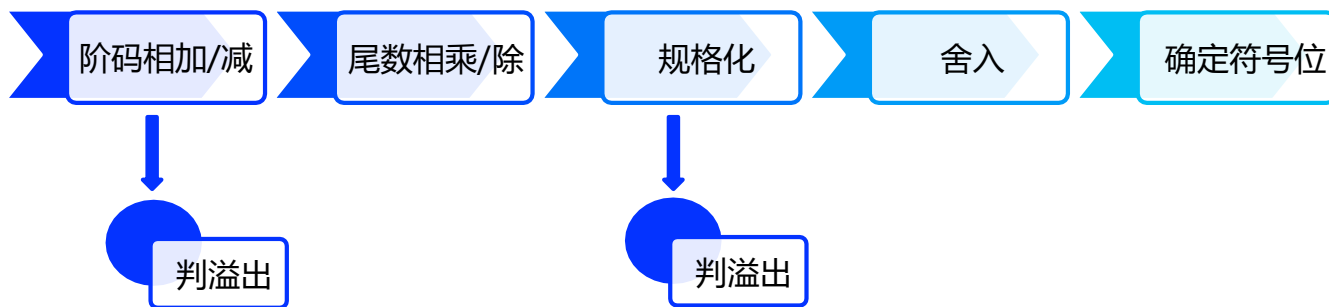
浮点乘法

■ 假设两个浮点数: $A = M_a \cdot 2^{E_a}$, $B = M_b \cdot 2^{E_b}$

$$A \times B = (M_a \times M_b) \cdot 2^{E_a + E_b}$$

$$A \div B = (M_a \div M_b) \cdot 2^{E_a - E_b}$$

计算机实现步骤:



浮点乘法——阶码相加

■ 阶码相加（单精度浮点乘法）

E_1, E_2 表示已带二进制的指数

$$\begin{aligned}[E_1 + E_2]_{\text{移}} &= 127 + E_1 + E_2 \\&= 127 + E_1 + 127 + E_2 - 127 \\&= [E_1]_{\text{移}} + [E_2]_{\text{移}} - 127 \\&= [E_1]_{\text{移}} + [E_2]_{\text{移}} + [-127]_{\text{补}} \\&= ([E_1]_{\text{移}} + [E_2]_{\text{移}} + 10000001\text{B}) \bmod 2^8\end{aligned}$$

$$([E_1]_{\text{移}} + [E_2]_{\text{移}} + 129) \bmod 2^8$$

浮点乘法——阶码相加

■例：若两个IEEE754操作数的阶码 分别为10和 - 5， 求 $10 + (-5)$ 的移码

解：

$$[E_1]_{\text{移}} = 127 + 10 = 137 = 1000\ 1001\text{B}$$

$$[E_2]_{\text{移}} = 127 + (-5) = 122 = 0111\ 1010\text{B}$$

$$\begin{aligned}[E_1 + E_2]_{\text{移}} &= [E_1]_{\text{移}} + [E_2]_{\text{移}} + 129 \\ &= (1000\ 1001 + 0111\ 1010 + 1000\ 0001) \pmod{2^8} \\ &= 1000\ 0100 = 132\end{aligned}$$

其阶码的和为 $132 - 127 = 5$ ，正好 等于 $10 + (-5) = 5$

浮点乘法——阶码相减

■ 阶码相减（单精度浮点除法）

$$\begin{aligned}[E_1 - E_2]_{\text{移}} &= E_1 - E_2 + 127 \\&= 127 + E_1 - (127 + E_2) + 127 \\&= [E_1]_{\text{移}} - [E_2]_{\text{移}} + 127 \\&= ([E_1]_{\text{移}} + [-[E_2]_{\text{移}}]_{\text{补}} + 01111111\text{B}) \bmod 2^8\end{aligned}$$

$$([E_1]_{\text{移}} + [-[E_2]_{\text{移}}]_{\text{补}} + 127) \bmod 2^8$$

浮点乘法——阶码相减

■例：若两个IEEE754操作数的阶码 分别为10和 - 5， 求 $10 - (-5)$ 的移码

解：

$$[E_1]_{\text{移}} = 127 + 10 = 137 = 1000\ 1001\text{B}$$

$$[E_2]_{\text{移}} = 127 + (-5) = 122 = 01111010\text{B}$$

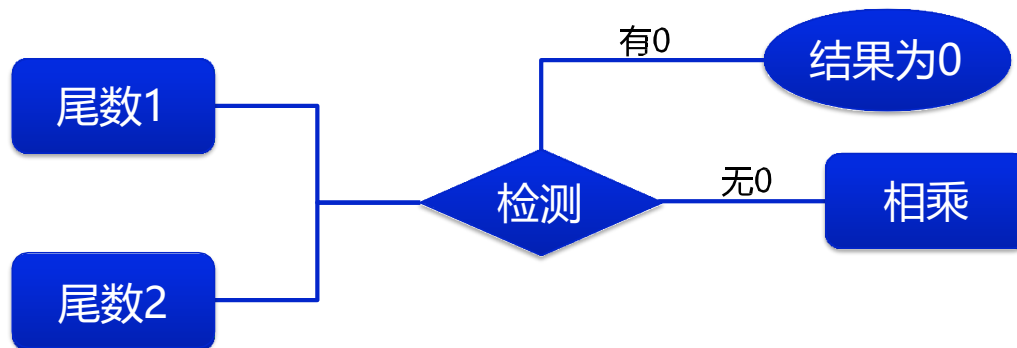
$$\begin{aligned}[E_1 - E_2]_{\text{移}} &= [E_1]_{\text{移}} + [-[E_2]_{\text{移}}]_{\text{补}} + 127 \\ &= (1000\ 1001 + 1000\ 0110 + 0111\ 1111) \pmod{2^8} \\ &= 1000\ 1110 = 142\end{aligned}$$

其阶码的差为 $142 - 127 = 15$ ，正 好等于 $10 - (-5) = 15$

浮点乘法——尾数运算

■ 尾数相乘

■ 预处理

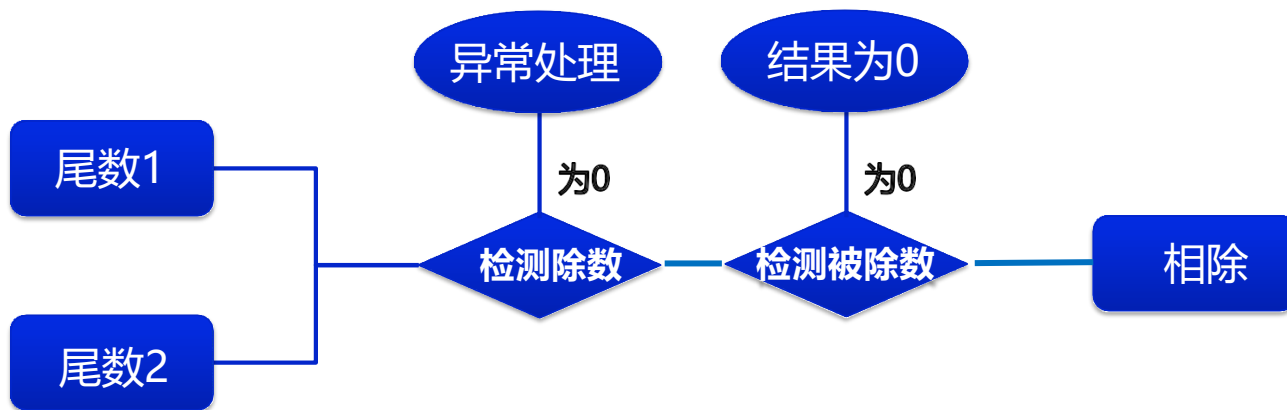


■ 相乘：定点小数原码乘法运算

浮点乘法——尾数运算

■ 尾数相除

■ 预处理



■ 相乘：定点小数原码除法运算

浮点乘法——规格化

■规格化原则

- 当尾数高位为0，左规
- 当尾数产生进位，右规
- 规格化尾数： $1 \leq \text{尾数}_1 < 2$ ， $1 \leq \text{尾数}_2 < 2$ （尾数形为1.xxx）

乘法： $1 \leq \text{尾数}_1 \times \text{尾数}_2 < 4$ 不需左规、最多右规一次

除法： $0.5 < \text{尾数}_1 \div \text{尾数}_2 < 2$ 不需右规、最多左规一次

浮点乘法——舍入&符号确定

■ 舍入

- 就近舍入
- 朝 $+\infty$ 舍入
- 朝 $-\infty$ 舍入
- 朝0舍入

若尾数是0，则需要将阶码也置0

■ 符号确定

- 同号：结果为正
- 异号：结果为负

浮点乘法——判定溢出

■ 阶码求和溢出

阶码求和

$$E_x + E_y + 129 = E_b$$

阶码求差

1xxxx 1xxxx
0xxxx 0xxxx

0xxxx 上溢
1xxxx 下溢

规格化

11111 上溢
00000 下溢

浮点乘法——判定溢出

■ 阶码求差溢出

阶码求和

$$E_x - E_y + 127 = E_b$$

阶码求差

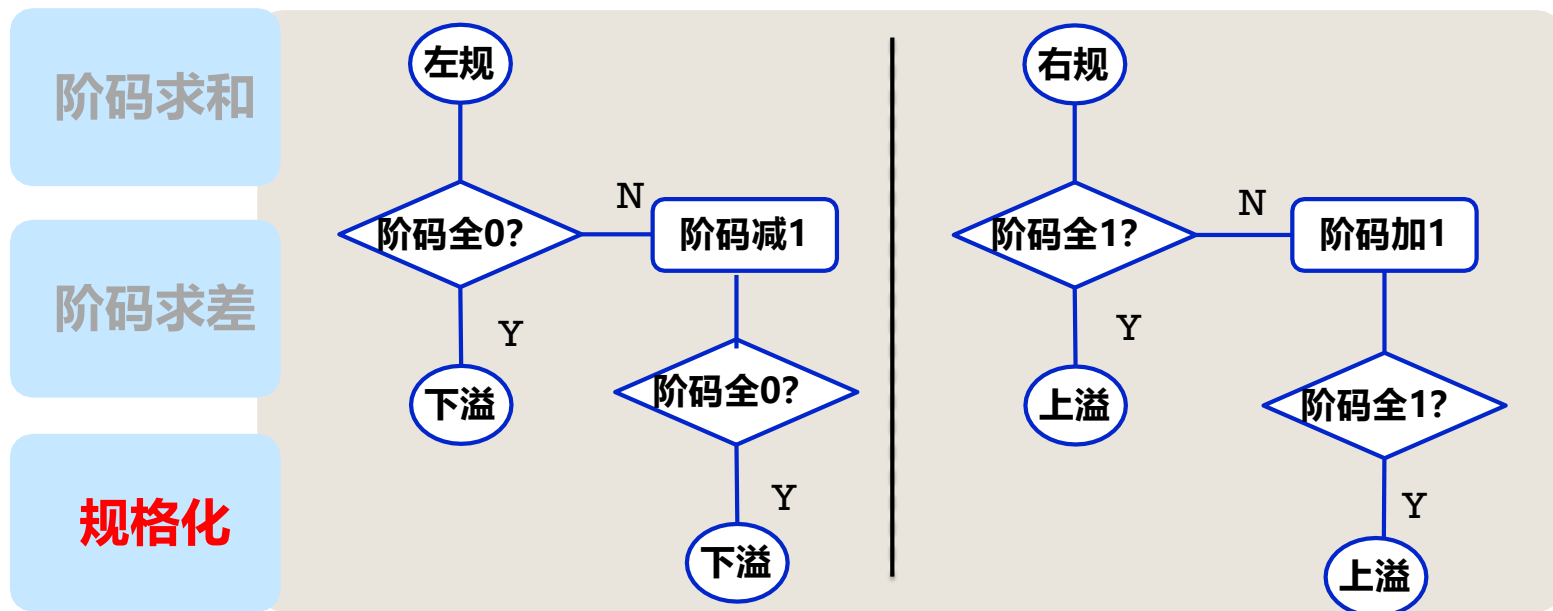
1xxxx	0xxxx	0xxxx	上溢
0xxxx	1xxxx	1xxxx	下溢

规格化

11111	上溢
00000	下溢

浮点乘法——判定溢出

■ 尾数规格化，阶码溢出



阶码溢出示例

例1：若 $E_b=0000\ 0001$ ，则尾数左规一次后，结果的阶码 $E_b=?$

解： $E_b = E_b + [-1]_{\text{补}} = 0000\ 0001 + 1111\ 1111$
 $= 0000\ 0000$ 阶码下溢！

例2：若 $E_x=1111\ 1110$ ， $E_y=1000\ 0000$ ，则乘法运算时，结果的阶码 $E_b=?$

解： $E_b = E_x + E_y + 129 = 1111\ 1110 + 1000\ 0000 + 1000\ 0001$
 $= 1111\ 1111$ 阶码上溢！

浮点数乘法示例

■ 例：用二进制的形式求出浮点数 0.5_{10} 与 -0.4375_{10} 之积（假设保留4位有效数位的精度）。

解： $0.5_{10} = 1/2_{10} = 0.1_2 = 1.000_2 \times 2^{-1}$

$-0.4375_{10} = -7/16_{10} = -0.0111_2 = -1.110_2 \times 2^{-2}$

◆ 阶码相加： $-1 + (-2) = -3$

◆ 尾数相乘： $1.000_2 \times 1.110_2 = 1.110000_2$

◆ 乘积为： $1.110000_2 \times 2^{-3}$

◆ 规格化并判溢出：

- 尾数 1.110000 已是规格化数, 且 $127 \geq -3 \geq -126$, 没有溢出!

◆ 舍入： $1.110_2 \times 2^{-3}$

◆ 确定符号位：乘积为负数, -1.110×2^{-3}

◆ 所以浮点乘法结果： $-1.110_2 \times 2^{-3} = -0.00111_2 = -7/32_{10} = -0.21875_{10}$

$$\begin{array}{r} 1.000_{\text{two}} \\ \times 1.110_{\text{two}} \\ \hline 0000 \\ 1000 \\ 1000 \\ 1000 \\ \hline 1110000_{\text{two}} \end{array}$$

浮点数除法示例

- 例：用二进制的形式求出浮点数 0.5_{10} 与 -0.4375_{10} 的商(假设保留4位有效 数位的精度)。
- 解： $0.5_{10} = 1/2_{10} = 0.1_2 = 1.000_2 \times 2^{-1}$ $-0.4375_{10} = -7/16_{10} = -0.0111_2 = -1.110_2 \times 2^{-2}$
- 阶码相减： $-1 - (-2) = 1$
- 尾数相除： $1.000_2 \div 1.110_2 = 0.1001001_2$, 商为： $0.1001001_2 \times 2^1$
- 规格化并判溢出： $1.0010010_2 \times 2^0$, $127 \geq 0 \geq -126$, 没有溢出
- 舍入： $1.001_2 \times 2^0$
- 确定符号位：商为负数, $-1.001_2 \times 2^0$
- 所以浮点除法结果：
 - $-1.001_2 \times 2^0 = -1.001_2 = -9/8_{10} = -1.125_{10}$

浮点数运算

浮点数运算：由多个ALU + 移位器实现

■ 加/减运算

- 对阶、尾数相加减、规格化处理、舍入、判断溢出

■ 乘/除运算

- 尾数用浮点原码乘/除运算实现，阶码相加/减运算实现

■ 溢出判断

- 当结果发生阶码上溢时，结果发生溢出；发生阶码下溢时，结果为0

■ 精确表示运算结果

- 中间结果增设保护位、舍入位，等
- 最终结果四种舍入方式：就近舍入 / 正向舍入 / 负向舍入 / 截去

MIPS中的浮点指令

■ 算术运算: *单精度*

■ Add.s ; add.d *双精度*

■ Sub.s ; sub.d

■ Mul.s ; mul.d

■ Div.s ; div.d

■ 比较: C.x.s ; c.x.d

■ X: eq, neq, lt, le, gt, ge

C → 协处理器

■ 分支: 根据浮点数标志位真假跳转

Bc1t , bc1f

■ 访存: 针对FPU (协处理器)

Lwcl ; swcl

■ 浮点寄存器: \$f0, \$f1,...

■ 双精度寄存器是偶数-奇数寄存器对

算术运算的精确性

■ 整数的精确表示问题

- 一定字长所能表示的整数范围是有限的
- 任何一个整数只要落在所能表示的最大值和最小值之间，就可以精确表示

■ 浮点数的精确表示问题

- 有些浮点数是无法用有限的字长精确地表示出来，一般只能用近似值代（舍入）
- 在任何两个实数(如0~1)之间都有无穷多个实数，对于单精度的浮点数所能表示的实数的总个数只有区区的 2^{24} 个

谬误和陷阱

由于计算机算术的有限精度与自然算术的无限精度之间的差异引起谬误与陷阱

谬误1：对于整数，左移指令相当于将原数与2的幂次方相乘；右移指令则相当于将原来的整数除以2的幂次方。

例：若将 -5除以4，正确的商应该是 -1。

– 5用二进制补码表示为：

$(1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1011)_2$

如果根据上述错误观点，除以4(2^2)就变成了**逻辑右移**2位，其结果为：

$(0011\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110)_2$

现在连符号位都变成了0，这样一个结果显然是错误的。右移得到的结果

实际上是 1073741822，而不是 $(-1)_{10}$ 。

如果使用**算术右移**指令，在右移时自动进行符号扩展，而非简单地将空出的最高位填0。这样，用算术右移指令将 $(-5)_{10}$ 右移2位后，得到结果为：

$(1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110)_2$

这样得到的是 $(-2)_{10}$ ，而不是 $(-1)_{10}$ 。

谬误和陷阱

谬误2：浮点加法满足结合律，即 $x+(y+z)=(x+y)+z$

例： $x = -1.5_{\text{ten}} \times 10^{38}$, $y = 1.5_{\text{ten}} \times 10^{38}$, 而 $z=1.0$ (且三个数都是单精度), 则

$$\text{有: } x+(y+z) = -1.5_{\text{ten}} \times 10^{38} + (1.5_{\text{ten}} \times 10^{38} + 1.0)$$

$$= -1.5_{\text{ten}} \times 10^{38} + (1.5_{\text{ten}} \times 10^{38}) = 0.0$$

$$(x+y)+z = (-1.5_{\text{ten}} \times 10^{38} + 1.5_{\text{ten}} \times 10^{38}) + 1.0$$

$$= (0.0_{\text{ten}}) + 1.0 = 1.0$$

因此，浮点加法不满足结合定律！

谬误和陷阱

谬误3：只有数学家才需要关注浮点数的精度问题

1994年11月，Pentium处理器浮点瑕疵问题，给 Intel公司的声誉带来严重损害，并造成了5亿美元的巨额损失。

爱国者导弹防御系统，曾因浮点数误差导致弹道预测发生严重错误，误差的累积演变成了生死攸关的灾难。